Distributed Systems
2026WS

universität innsbruck

703088 (2026WS) - PS Distributed Systems
Distributed and Parallel Systems Group
University of Innsbruck
Instructors: Juan Aznar Poveda, Zahra Najafabadi Samani, and Marlon Etheredge

EX04 - Leader Election

Introduction

In the previous exercises, you explored peer-to-peer discovery and communication using Zenoh. While communication primitives allow nodes to discover and talk to each other, distributed applications frequently need to move beyond simple message exchange to achieve coordination and consistency.

Coordination, Symmetry, and the Need for a Leader

In a purely symmetric peer-to-peer or mesh network, all nodes are equal participants. While this decentralization offers high resilience and eliminates single points of failure, it introduces a severe coordination challenge: how can independent nodes agree on a single course of action, state update, or shared resource assignment without conflicting?

Distributed systems literature addresses this through synchronization and consistency models:

- **Symmetric/Leaderless Coordination:** Nodes use techniques like vector clocks, quorum reads/writes, or conflict-free replicated data types (CRDTs). While fault-tolerant, these approaches often complicate state management or incur high overhead for linearizable operations.
- **Asymmetric/Leader-Based Coordination:** To simplify decision-making and guarantee strict linearizability (e.g., in distributed databases or transaction logs), the system dynamically selects a single coordinator, or leader, to order operations and arbitrate state changes.

Leader Election in Distributed Literature and Industry

To automate the selection of a coordinator, distributed systems rely on **leader election algorithms**. Depending on the architecture, different strategies are used to determine who takes charge:

- **The Bully Algorithm:** Relies on node identifiers. When a node detects that the leader has failed, it "bullies" nodes with lower IDs by broadcasting an election message. The node with the highest active identifier ultimately claims leadership. While straightforward, it can generate heavy messaging overhead and performs poorly during network partitions.
- **Ring-Based Algorithms (e.g., Chang-Roberts):** Nodes are arranged in a logical ring, passing a token or election message sequentially around the loop to find the maximum identifier. This approach optimizes message complexity but is slower and sensitive to link disruptions.
- **Randomized Timeout Elections:** Modern consensus-driven systems use randomized heartbeats and timeout phases (similar to the mechanisms found in Paxos or Raft variants) to organically settle elections and prevent perpetual split votes in dynamic clusters.

Real-World Applications and Industry Systems

Leader election is a cornerstone of modern infrastructure software:

- **Distributed Databases & Coordination Services:** Systems like **Apache ZooKeeper**, **etcd**, and **Consul** use leader election to establish an active controller that handles writes, maintains metadata, and coordinates cluster membership.
- **Stream Processing Engines:** Frameworks like **Apache Kafka** rely on an internal leader (the Controller or partition leader) to manage topic partition assignments and track consumer offsets.

- **High-Availability Services:** Cloud native orchestrators and distributed cron schedulers elect a single master node to ensure periodic tasks or state transitions are executed exactly once, preventing race conditions.

Once a dynamic mesh of peers has discovered each other, the system must transition from raw connectivity to coordinated control. Designating a leader manually is impossible in an environment where nodes can join, leave, or fail arbitrarily.

The system itself must autonomously agree on who is in charge. This exercise introduces the foundational mechanics of state-based leader election using timeouts, terms, and states:

1. **Follower:** Passive state; responds to incoming messages. If no communication is received within an election timeout, it transitions to a candidate.
2. **Candidate:** Initiates a new election term, votes for itself, and broadcasts election requests to all peers.
3. **Leader:** Periodically sends heartbeats to maintain authority and suppress new elections.

> **How can a distributed system autonomously and safely choose a single leader without a centralized coordinator?**

Prerequisites

You should have: - basic knowledge of distributed state machines and network RPC messaging.

For this exercise, use a programming language of your choice to implement a leader election state machine.

> **Note:** You only need to implement the election and heartbeat mechanics. Full log replication and state machine execution are out of scope for this exercise.

Task 1 - State Machine & Leader Election Implementation

Implement a distributed program that runs on at least three distinct nodes (A, B, and C).

- **1.1).** Maintain the following persistent and volatile state variables on every node:
  - currentTerm: monotonically increasing term counter (initialized to 0).
  - votedFor: node ID that received a vote in the current term (or null).
  - state: current role (Follower, Candidate, or Leader).
- **1.2).** Implement randomized election timeouts (e.g., between 150ms and 300ms) for Followers and Candidates:
  - If a Follower's timer expires without a heartbeat, transition to Candidate, increment currentTerm, vote for self, and broadcast a RequestVote RPC.
  - Upon receiving votes from a majority (e.g., 2 out of 3 nodes), transition immediately to Leader and begin sending periodic heartbeats (e.g., every 50ms).

Task 2 - Experiments & Failure Recovery

Run three node instances concurrently in separate terminals and record your observations:

- **2.1).** Initial Leader Election: Start Nodes A, B, and C. Verify that a single node is elected leader and that all followers receive its periodic heartbeats.
- **2.2).** Leader Crash & Re-election: Stop the active leader process. Observe and record:
  - How remaining nodes detect the failure.
  - The transition of a follower to candidate.
  - The outcome of the new election term.
- **2.3).** Node Recovery: Restart the previously stopped node. Observe how its outdated term is updated and how it reintegrates as a follower.

Task 3 - Conceptual Questions

- **3.1).** Answer the following questions concise and precisely:
  - Why are randomized election timeouts critical for preventing perpetual split votes?
  - What happens if two candidates receive an equal number of votes in a cluster, and how does the system recover?

  - How does comparing currentTerm prevent an isolated or delayed node from disrupting the active cluster with stale messages?
  - Is leader election alone sufficient to guarantee linearizable consistency in a distributed database? Explain briefly.

Submission

Submit a PDF file with your solution and your code. You can optionally use the markdown template provided in OLAT (`Submission_Template.md`) and convert it to PDF.

Your report should address all tasks and questions specified in the exercise. Depending on the exercise, this may include:

- theoretical questions and explanations;
- a description of your implementation;
- source code snippet;
- experimental results, observations, or screenshots;
- answers to the discussion questions.

Focus your report on the **distributed-systems concepts, your design decisions, and your observations**. Keep the report concise and avoid documenting routine installation steps unless they are relevant to the exercise.

The submission should follow the following structure and naming convention:

```
|-- exc0X_report_csXXXXXX.pdf
`-- exc0X_source_csXXXXXX.zip
    |-- README.md
    |-- sample_file_a.py
    |-- sample_file_b.py
    `-- sample_file_c.py
```

Deadline

12.11.2026 12:00h

Presentation

13.10.2026